TradeLearn

Learn skills. Build real values.

Source Code Submission

(First and Last Portions of Source Code)

Prepared for Copyright Submission

Author:

Rajvardhan Tiwari

File: backend/app.js

```
const express = require("express")
const app = express()

const session = require("express-session");
const loginModel = require("./models/login") // user data handle
const messageModel = require("./models/message") // messaging system

const cookieparser = require("cookie-parser");
const path = require('path');
const fs = require("fs");
const otpStorage = require('./storage'); // OTP temporary store

const jwt = require("jsonwebtoken"); // authentication
const multer = require("multer"); // file upload

require('dotenv').config();

const register = require('./routes/register')
const login = require('./routes/login')
const otpVerification = require('./routes/otpVerification')
const transporter = require('./transporter') // email sending

app.use(cookieparser())
app.use(express.json())
app.use(express.urlencoded({extended:true}))
app.use(express.static(path.join(__dirname,'public'))) // static files serve

app.set('views', path.join(__dirname, 'views'))
app.set('view engine', 'ejs') // template engine

app.use(session({
    secret: 'yourSecretKey',
    resave: false,
    saveUninitialized: true
}))

// file upload configuration
const storage = multer.diskStorage({
    destination: (req, file, cb) => cb(null, "public/uploads/"),
    filename: (req, file, cb) => cb(null, Date.now() +
path.extname(file.originalname))
})

// only image files allowed
const fileFilter = (req, file, cb) => {
    if (file.mimetype.startsWith("image/")) cb(null, true)
    else cb(new Error("Only images allowed"), false)
}

const upload = multer({ storage, fileFilter })
const defaultProfilePic = "dummyUser.jpg"

// delete old profile picture
```

```
function removeUploadedFile(filename) {
    if (!filename || filename === defaultProfilePic) return

    const filePath = path.join(__dirname, "public", "uploads", filename)
    if (fs.existsSync(filePath)) fs.unlinkSync(filePath)
}

// count unread messages
async function getUnreadConversationCount(username) {
    const chats = await messageModel.find({ from: username }).select("content")

    return chats.reduce((count, chat) => {
        const last = chat.content?.[chat.content.length - 1] || ""
        return (!last.startsWith(username + ":")) ? count + 1 : count
    }, 0)
}

// update profile picture
app.post("/update-profile-pic", isLoggedIn, upload.single("profilePic"), async (req,
res) => {
    if (!req.file) return res.redirect("/dashboard")

    const user = await loginModel.findOne({ username: req.user.username })
    if (!user) return res.send("User not found")

    await loginModel.findOneAndUpdate(
        { username: req.user.username },
        { profilePic: req.file.filename }
    )

    removeUploadedFile(user.profilePic)
    res.redirect("/dashboard")
})

// remove profile picture
app.post("/remove-profile-pic", isLoggedIn, async (req, res) => {
    const user = await loginModel.findOne({ username: req.user.username })

    removeUploadedFile(user.profilePic)

    await loginModel.findOneAndUpdate(
        { username: req.user.username },
        { profilePic: defaultProfilePic }
    )

    res.redirect("/dashboard")
})

app.get("/", (req,res)=> res.render('login'))

app.use("/register", register)
app.use("/login", login)
app.use("/otpVerification", otpVerification)

// search users by skill
```

```
app.post("/footer", isLoggedIn, async (req, res) => {
    let search = (req.body.search || "").trim()

    let result = await loginModel.find({
        skill: { $regex: search, $options: "i" },
        username: { $ne: req.user.username }
    })

    res.render("footer", { result, hasSearched: true, searchTerm: search })
})

// show messages list
app.get("/PersonalMessages", isLoggedIn, async (req,res)=>{
    let user = req.user.username

    let data = await messageModel.find({from:user}).sort({lastUpdated:-1})
    const unreadCount = await getUnreadConversationCount(user)

    res.render('PersonalMessages',{data, currentUser:user, unreadCount})
})

// send message
app.post("/text/:username", isLoggedIn, async (req, res) => {
    let from = req.user.username
    let to = req.params.username

    let text = (req.body.text || "").trim()
    if (!text) return res.redirect(`/text/${to}`)

    let msg = `${from}: ${text}`

    await messageModel.findOneAndUpdate(
        { from, to },
        { $push: { content: msg }, $set: { lastUpdated: Date.now() } },
        { upsert: true }
    )

    await messageModel.findOneAndUpdate(
        { from: to, to: from },
        { $push: { content: msg }, $set: { lastUpdated: Date.now() } },
        { upsert: true }
    )

    res.redirect(`/text/${to}`)
})

// authentication middleware
function isLoggedIn(req,res,next){
    if (!req.cookies.token) return res.send("Not authorized")

    try {
        req.user = jwt.verify(req.cookies.token, 'secret-key')
        next()
    } catch {
        return res.send("Invalid token")
```

```
    }
}

// 404 handler
app.use((req, res) => {
    res.status(404).render('Error')
})

app.listen(3000,()=> console.log("Server running"))
```

File: backend/storage.js

```
const otpStorage = {}; // empty object to temporarily store OTPs (key = email, value
= OTP)

module.exports = otpStorage; // export this object so other files can access and use
same OTP storage
```

File: backend/transporter.js

```
const nodemailer = require("nodemailer"); // library used to send emails

// create transporter (configuration for sending emails)
const transporter = nodemailer.createTransport({
    service: 'gmail', // using Gmail service
    host: 'smtp.gmail.com', // Gmail SMTP server
    port: 587, // port for TLS
    secure: false, // false because using TLS, not SSL
    type: "login", // authentication type
    auth: {
        user: "ushabhjain07@gmail.com", // sender email address
        pass: "vhir zwfa pslk sdse" // app password for authentication
    }
});

module.exports = transporter; // export transporter to use in other files
```

File: backend/models/login.js

```
// Import mongoose to interact with MongoDB
const mongoose = require("mongoose");

// Load environment variables
require('dotenv').config();


// Establish connection with MongoDB database
mongoose.connect("mongodb://localhost:27017", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log("Connected to MongoDB"))
.catch((error) => console.error("MongoDB connection error:", error));


// Define schema for user data (structure of user collection)
const loginSchema = mongoose.Schema({
  name: { type: String },
  username: { type: String },
  skill: { type: String },
  email: { type: String },
  github: { type: String },
  linkedin: { type: String },
  profilePic: { type: String, default: "dummyUser.jpg" },
  password: { type: String },
}, {
  timestamps: true // adds createdAt and updatedAt automatically
});


// Create and export model to interact with "login" collection
module.exports = mongoose.model('login', loginSchema);
```

File: backend/models/message.js

```
// Import mongoose to define schema and interact with MongoDB
const mongoose = require("mongoose");

// (This line is unnecessary here — can be removed)
const { type } = require("os");


// Define schema for storing messages between users
const messageSchema = mongoose.Schema({
    to: { type: String, required: true }, // receiver username
    from: { type: String, required: true }, // sender username

    // array storing conversation messages (each entry = one message)
    content: [{
        type: String
```

```
    }],

    lastUpdated: { type: Date, default: Date.now } // last message timestamp
});


// Create and export model for "messages" collection
module.exports = mongoose.model('messages', messageSchema);
```

File: backend/routes/login.js

```
// Import required modules and dependencies
const express = require('express');
const router = express.Router();
const loginModel = require("../models/login"); // user model for DB operations

const bcrypt = require("bcrypt"); // for password comparison
const jwt = require("jsonwebtoken"); // for generating auth token


// Login route (handles user authentication)
router.post('/', async (req, res) => {
    let { username, password } = req.body;

    username = username.toLowerCase(); // normalize username

    // Find user in database
    let user = await loginModel.findOne({ username });

    if (!user) {
        return res.status(500).render("LoginError"); // user not found
    }

    // Compare entered password with hashed password in DB
    bcrypt.compare(password, user.password, (err, result) => {
        if (result) {

            // Generate JWT token for authenticated user
            let token = jwt.sign(
                { username: username, userid: user._id },
                'secret-key'
            );

            res.cookie('token', token); // store token in cookies

            // Redirect to OTP verification step

res.status(200).redirect(`/loginOTP?email=${user.email}&name=${user.name}`);

        } else {
            res.render('LoginError'); // incorrect password
        }
    });
});
```

```
// Export router to use in main app
module.exports = router;
```

File: backend/routes/otpVerification.js

```
// Import required modules
const express = require('express');
const router = express.Router();
const loginModel = require("../models/login"); // user model

const bcrypt = require("bcrypt"); // password hashing
const jwt = require("jsonwebtoken"); // token generation

const otpStorage = require('../storage'); // temporary OTP storage


// Route to verify OTP and complete registration
router.post('/', async (req, res) => {
    const { email, otp } = req.body;

    // Check if OTP is valid
    const storedOtp = otpStorage[email];
    if (!storedOtp || storedOtp !== otp) {
        return res.status(400).send("Invalid OTP.");
    }

    delete otpStorage[email]; // remove OTP after verification


    // Get registration data stored in session
    const registrationData = req.session.registrationData;
    if (!registrationData) {
        return res.status(400).send("Registration data missing.");
    }

    let { name, username, skill, password, github, linkedin } = registrationData;
    username = username.toLowerCase();

    // Check if user already exists
    let user = await loginModel.findOne({ username });
    if (user) {
        return res.status(500).send("User Already Registered");
    }


    // Hash password and create new user
    const saltRounds = 10;
    bcrypt.genSalt(saltRounds, (err, salt) => {
        bcrypt.hash(password, salt, async (err, hash) => {

            let user = await loginModel.create({
                name,
                username,
                skill,
```

```
                email,
                github,
                linkedin,
                password: hash
            });

            // Generate JWT token after successful registration
            let token = jwt.sign(
                { username: username, userid: user._id },
                'secret-key'
            );

            res.cookie('token', token); // store token
            res.render('HomePage', { name: name }); // redirect to homepage
        });
    });
});


// Export router
module.exports = router;
```

File: backend/routes/register.js

```
// Import required modules
const express = require('express');
const router = express.Router();

const otpStorage = require('../storage'); // temporary OTP storage
require('dotenv').config();

const transporter = require("../transporter"); // email sender


// Registration route (stores user data and sends OTP)
router.post('/', (req, res) => {
    const { name, username, skill, email, github, linkedin, password } = req.body;

    // Save registration data in session (used after OTP verification)
    req.session.registrationData = { name, username, skill, email, github, linkedin,
password };

    // Generate 6-digit OTP
    const otp = Math.floor(100000 + Math.random() * 900000).toString();

    // Store OTP against user's email
    otpStorage[email] = otp;

    // Email configuration for sending OTP
    const mailOptions = {
        from: 'your-email@example.com',
        to: email,
        subject: 'Your OTP for 2-Factor Authentication',
        text: `Your OTP is ${otp}. It is valid for 5 minutes.`
    };
```

```
    // Send OTP email
    transporter.sendMail(mailOptions, (error, info) => {
        if (error) {
            return res.status(500).send('Error in sending OTP');
        }

        // Redirect to OTP verification page
        res.redirect(`/otpVerification?email=${email}`);
    });
});


// Export router
module.exports = router;
```

Files: Frontend/views/login.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
    <!-- Basic page setup and responsiveness -->
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>TradeLearn System - Login</title>

    <!-- Tailwind CSS for styling -->
    <script src="https://cdn.tailwindcss.com"></script>

    <!-- Custom CSS file -->
    <link rel="stylesheet" href="../public/css/login.css">

    <!-- Custom animations -->
    <style>
        @keyframes slideIn {
            from {
                transform: translateY(20px);
                opacity: 0;
            }
            to {
                transform: translateY(0);
                opacity: 1;
            }
        }

        @keyframes pulse {
            0%, 100% {
                transform: scale(1);
            }
            50% {
                transform: scale(1.05);
            }
        }
```

```
        .animate-slideIn {
            animation: slideIn 0.6s ease-out;
        }

        .animate-pulse {
            animation: pulse 2s infinite;
        }
    </style>
</head>

<body>
    <!-- Main container with centered login card -->
    <div class="w-full h-screen bg-gradient-to-r from-zinc-900 to-zinc-800 text-white
flex items-center justify-center">

        <!-- Login card -->
        <div class="p-10 w-full max-w-md bg-zinc-800 rounded-lg shadow-lg animate-
slideIn">

            <!-- Heading and description -->
            <h1 class="text-4xl my-10 font-semibold text-center">Welcome to
TradeLearn</h1>
            <p class="text-center mb-6">Exchange skills, learn, and grow together.
Login to explore opportunities!</p>

            <!-- Login form (sends data to backend /login route) -->
            <form action="/login" method="post">

                <!-- Username input -->
                <input
                    class="block mt-3 w-full px-3 py-2 rounded-md bg-transparent
border-2 border-zinc-700 outline-none focus:ring-2 focus:ring-blue-500"
                    type="text"
                    placeholder="Username"
                    name="username"
                    required
                />

                <!-- Password input -->
                <input
                    class="block mt-3 w-full px-3 py-2 rounded-md bg-transparent
border-2 border-zinc-700 outline-none focus:ring-2 focus:ring-blue-500"
                    type="password"
                    placeholder="Password"
                    name="password"
                    required
                />

                <!-- Submit button -->
                <input
                    class="block mt-5 px-5 py-3 bg-blue-500 rounded-md hover:bg-blue-
600 cursor-pointer transition-all animate-pulse"
                    type="submit"
                    value="Login"
                />
```

```
            </form>

            <!-- Redirect to register page -->
            <a class="mt-10 text-blue-500 block text-center hover:underline"
href="/Register">
                New here? Create an account
            </a>

        </div>
    </div>
</body>
</html>
```

Files: Frontend/views/index.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
    <!-- Basic page setup and responsive design -->
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>TradeLearn System - Register</title>

    <!-- Tailwind CSS for styling -->
    <script src="https://cdn.tailwindcss.com"></script>

    <!-- Custom animations and styles -->
    <style>
        @keyframes slideIn {
            from {
                transform: translateY(20px);
                opacity: 0;
            }
            to {
                transform: translateY(0);
                opacity: 1;
            }
        }

        @keyframes pulse {
            0%, 100% { transform: scale(1); }
            50% { transform: scale(1.05); }
        }

        .animate-slideIn {
            animation: slideIn 0.6s ease-out;
        }

        .animate-pulse {
            animation: pulse 2s infinite;
        }

        /* Adds red * for required fields */
        .required::after {
```

```
            content: " *";
            color: red;
        }
    </style>
</head>

<body>
    <!-- Main container (centered form with background gradient) -->
    <div class="w-full md:h-screen sm:h-auto bg-gradient-to-r from-zinc-900 to-zinc-
800 text-white flex items-center justify-center">

        <!-- Registration card -->
        <div class="p-5 w-full max-w-2xl bg-zinc-800 rounded-lg shadow-lg animate-
slideIn">

            <!-- Heading and intro text -->
            <h1 class="text-2xl md:text-4xl font-semibold text-center">Join
TradeLearn</h1>
            <p class="text-center mb-4 md:mb-6 text-base md:text-lg">
                Create an account and start exchanging skills with others.
            </p>

            <!-- Registration form (sends data to backend /register route) -->
            <form action="/register" method="post" class="flex flex-wrap -mx-2">

                <!-- User basic details -->
                <div class="w-full md:w-1/2 px-2">
                    <input id="name" class="block mt-1 w-full px-3 py-2 rounded-md
bg-transparent border-2 border-zinc-700 outline-none focus:ring-2 focus:ring-blue-
500"
                           type="text" placeholder="Name" name="name" required />
                </div>

                <div class="w-full md:w-1/2 px-2">
                    <input id="username" class="block mt-1 w-full px-3 py-2 rounded-
md bg-transparent border-2 border-zinc-700 outline-none focus:ring-2 focus:ring-blue-
500"
                           type="text" minlength="5" placeholder="Username"
name="username" required />
                </div>

                <div class="w-full md:w-1/2 px-2">
                    <input id="skill" class="block mt-1 w-full px-3 py-2 rounded-md
bg-transparent border-2 border-zinc-700 outline-none focus:ring-2 focus:ring-blue-
500"
                           type="text" placeholder="Skill" name="skill" required />
                </div>

                <div class="w-full md:w-1/2 px-2">
                    <input id="email" class="block mt-1 w-full px-3 py-2 rounded-md
bg-transparent border-2 border-zinc-700 outline-none focus:ring-2 focus:ring-blue-
500"
                           type="email" placeholder="Email" name="email" required />
                </div>
```

```
                <!-- Optional social links -->
                <div class="w-full md:w-1/2 px-2">
                    <input id="github" class="block mt-1 w-full px-3 py-2 rounded-md
bg-transparent border-2 border-zinc-700 outline-none focus:ring-2 focus:ring-blue-
500"
                           type="text" placeholder="GitHub" name="github" />
                </div>

                <div class="w-full md:w-1/2 px-2">
                    <input id="linkedin" class="block mt-1 w-full px-3 py-2 rounded-
md bg-transparent border-2 border-zinc-700 outline-none focus:ring-2 focus:ring-blue-
500"
                           type="text" placeholder="LinkedIn" name="linkedin" />
                </div>

                <!-- Password with validation rules -->
                <div class="w-full px-2">
                    <input id="password" class="block mt-1 w-full px-3 py-2 rounded-
md bg-transparent border-2 border-zinc-700 outline-none focus:ring-2 focus:ring-blue-
500"
                        type="password"
                        placeholder="Password"
                        name="password"
                        required
                        minlength="8"
                        pattern="^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*])[A-
Za-z\d!@#$%^&*]{8,}$"
                        title="Password must contain uppercase, lowercase, number and
special character"
                    />
                </div>

                <!-- Submit button -->
                <div class="w-full px-2">
                    <input class="block mt-5 px-5 py-3 bg-blue-500 rounded-md
hover:bg-blue-600 cursor-pointer transition-all animate-pulse"
                           type="submit" value="Register" />
                </div>
            </form>

            <!-- Redirect to login page -->
            <a class="mt-10 text-blue-500 block text-center hover:underline"
href="/">
                Already a member? Sign in
            </a>
        </div>
    </div>
</body>
</html>
```

File: views/HomePage.ejs

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Register</title>

    <!-- Tailwind CSS -->
    <script src="https://cdn.tailwindcss.com"></script>
</head>

<body>
    <!-- Registration form (sends data to /register route) -->
    <form action="/register" method="post">

        <!-- User details -->
        <input type="text" name="name" placeholder="Name" required />
        <input type="text" name="username" placeholder="Username" required />
        <input type="text" name="skill" placeholder="Skill" required />
        <input type="email" name="email" placeholder="Email" required />

        <!-- Optional links -->
        <input type="text" name="github" placeholder="GitHub" />
        <input type="text" name="linkedin" placeholder="LinkedIn" />

        <!-- Password input -->
        <input type="password" name="password" placeholder="Password" required />

        <!-- Submit -->
        <input type="submit" value="Register" />
    </form>

    <!-- Redirect to login -->
    <a href="/">Already have an account? Login</a>

</body>
</html>
```

File: Frontend/views/aboutus.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>About Us - Skill Learning Through Connections</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.6.0/css/all.min.css" integrity="sha512-
Kc323vGBEqzTmouAECnVceyQqyqdsSiqLQISBL29aUW4U/M7pSPA/gEUZQqv1cwx4OnYxTxve5UMg5GT6L4JJ
g==" crossorigin="anonymous" referrerpolicy="no-referrer" />
</head>
<body class="bg-gray-100 text-gray-800">
    <div class="absolute top-4 left-4">
        <button class="bg-blue-500 hover:bg-blue-600 text-white font-bold py-2 px-4
rounded">
            <a href="javascript:history.back()">Back</a>
```

```
        </button>
    </div>


    <header class="bg-zinc-900 text-white py-12 text-center">
        <div class="container mx-auto">
            <h1 class="text-4xl font-bold font-sans">About Us</h1>
            <p class="text-lg mt-4 font-mono">Empowering Skill Development Through
Online Connections</p>
        </div>
    </header>

    <section class="py-12 bg-white">
        <div class="container mx-auto flex flex-wrap items-center justify-between px-
4">
            <div class="w-full md:w-1/2 p-4">
                <h2 class="text-3xl font-bold mb-6 font-sans">Our Mission</h2>
                <p class="text-lg leading-relaxed font-mono">We believe in the power
of community and connection. Our mission is to provide a platform where people from
all walks of life can meet online, share their knowledge, and learn new skills
together. Whether you're a seasoned professional or just starting out, there's always
something new to learn when you connect with others.</p>
            </div>
            <div class="w-full md:w-1/2 p-4 text-center">
                <img src="/images/learnigtogether.avif" alt="Learning Together"
class="w-full max-w-lg mx-auto rounded-lg">
            </div>
        </div>
    </section>

    <section class="py-12 bg-gray-200">
        <div class="container mx-auto px-4">
            <h2 class="text-3xl font-bold text-center mb-8 font-sans">Our Values</h2>
            <div class="flex flex-wrap gap-8 justify-center">
                <div class="bg-white p-6 rounded-lg shadow-md text-center w-full
sm:w-1/3">
                    <img src="/images/community.avif" alt="Community" class="w-40 mx-
auto mb-4">
                    <h3 class="text-xl font-semibold mb-2">Community</h3>
                    <p class="text-lg leading-relaxed">We foster a supportive and
inclusive environment where everyone is encouraged to share their knowledge and learn
from others.</p>
                </div>
                <div class="bg-white p-6 rounded-lg shadow-md text-center w-full
sm:w-1/3">
                    <img src="/images/growth.avif" alt="Growth" class="w-40 mx-auto
">
                    <h3 class="text-xl font-semibold mb-2">Growth</h3>
                    <p class="text-lg leading-relaxed">Personal and professional
growth is at the heart of everything we do. We help you unlock your potential by
connecting you with experts and learners worldwide.</p>
                </div>
                <div class="bg-white p-6 rounded-lg shadow-md text-center w-full
sm:w-1/3">
                    <img src="/images/collab.avif" alt="Collaboration" class="w-40
```

```
mx-auto mb-4">
                    <h3 class="text-xl font-semibold mb-2">Collaboration</h3>
                    <p class="text-lg leading-relaxed">We believe that collaboration
leads to innovation. By working together, we can achieve more and learn faster.</p>
                </div>
            </div>
        </div>
    </section>

    <footer class="bg-zinc-900 text-white py-6">
        <div class="container mx-auto text-center">
            <p class="mb-4">© 2024 TradeLearn Platform. All Rights Reserved.</p>
            <div class="flex justify-center space-x-6 text-2xl">
                <a href="#" class="text-blue-400 hover:text-blue-600"><i class="fab
fa-facebook-f"></i></a>
                <a href="#" class="text-blue-400 hover:text-blue-600"><i class="fab
fa-twitter"></i></a>
                <a href="#" class="text-blue-400 hover:text-blue-600"><i class="fab
fa-linkedin-in"></i></a>
                <a href="#" class="text-blue-400 hover:text-blue-600"><i class="fab
fa-instagram"></i></a>
            </div>
        </div>
    </footer>
</body>
</html>
```

File: views/dashboard.ejs

```
<!DOCTYPE html>
<html lang="en">
<head>
    <!-- Basic setup + responsive layout -->
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>User Dashboard</title>

    <!-- Tailwind CSS for styling -->
    <script src="https://cdn.tailwindcss.com"></script>

    <!-- Font Awesome for icons -->
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.6.0/css/all.min.css">
</head>

<body class="min-h-screen bg-gradient-to-br from-zinc-900 via-zinc-800 to-slate-900
text-white flex items-center justify-center p-4">

    <!-- Decide which profile image to show (uploaded or default) -->
    <% const profileImageSrc = result.profilePic && result.profilePic !==
"dummyUser.jpg"
        ? "/uploads/" + result.profilePic
        : "/images/dummyUser.jpg"; %>

    <!-- Navigation buttons -->
```

```
    <div class="absolute top-4 left-4">
        <a href="/home">Home</a>
    </div>

    <div class="absolute top-4 right-4">
        <a href="/logout">Logout</a>
    </div>

    <!-- Main dashboard container -->
    <main>

        <!-- Profile section -->
        <section>

            <div>
                <!-- Profile image -->
                <img
                    id="uploadedImagePreview"
                    src="<%= profileImageSrc %>"
                    alt="Profile Picture"
                />

                <!-- Upload new profile picture -->
                <form action="/update-profile-pic" method="POST"
enctype="multipart/form-data" id="profileUploadForm">
                    <input type="file" name="profilePic" id="profilePic"
accept="image/*" />
                </form>

                <!-- Remove profile picture -->
                <form action="/remove-profile-pic" method="POST">
                    <button type="submit">Remove</button>
                </form>
            </div>

            <!-- User basic info -->
            <h2><%= result.name %></h2>
            <p><%= result.email %></p>
        </section>

        <!-- User details -->
        <section>
            <p>Skill: <%= result.skill %></p>

            <!-- Social links -->
            <% if (result.linkedin) { %>
                <a href="<%= result.linkedin %>" target="_blank">LinkedIn</a>
            <% } %>

            <% if (result.github) { %>
                <a href="<%= result.github %>" target="_blank">GitHub</a>
            <% } %>
        </section>

        <!-- Account actions -->
```

```
        <section>
            <a href="/updateuser">Update Account</a>
            <a href="/userdelete">Delete Account</a>
        </section>

    </main>

    <!-- Script for image preview + auto upload -->
    <script>
        const profilePicInput = document.getElementById("profilePic");
        const preview = document.getElementById("uploadedImagePreview");
        const uploadForm = document.getElementById("profileUploadForm");

        profilePicInput.addEventListener("change", (event) => {
            const file = event.target.files[0];
            if (!file) return;

            // Show preview before upload
            const reader = new FileReader();
            reader.onload = function (e) {
                preview.src = e.target.result;
            };
            reader.readAsDataURL(file);

            // Auto submit form after selecting file
            uploadForm.submit();
        });
    </script>

</body>
</html>
```

Files: frontend/views/footer.ejs

```
<!-- SEARCH (footer.ejs) -->
<!-- This section allows users to search other users based on their skills -->
<form action="/footer" method="post">
    <input type="text" name="search" placeholder="Search by skill" required />
    <input type="submit" value="Search" />
</form>

<!-- After user submits the search query, results are fetched and displayed -->
<% if (hasSearched) { %>
    <% result.forEach(user => { %>
        <p>
            Username: <%= user.username %> <br>
            Skill: <%= user.skill %>
        </p>
    <% }) %>
<% } %>

<p>
    This search feature improves user interaction by allowing users to quickly find
    others with specific skills. It connects the frontend form with backend logic
    and dynamically renders results using EJS templating.
```

```
</p>
```

Files: Frontend/views/PersonalMessages.ejs

```
<!-- Displays all conversations associated with the logged-in user -->
<% data.forEach(chat => { %>
    <a href="/text/<%= chat.to %>">
        <p>Chat with: <%= chat.to %></p>
    </a>
<% }) %>

<p>Total Unread Conversations: <%= unreadCount %></p>

<p>
    This section represents the messaging dashboard where users can view all their
    active conversations. It helps users easily navigate between chats and keeps
    track of unread messages for better communication management.
</p>
```

Files: Frontend/views/message.ejs
+

```
<!-- Form used to send a message to another user -->
<form action="/text/<%= username %>" method="post">
    <input type="text" name="text" placeholder="Enter your message" required />
    <input type="submit" value="Send Message" />
</form>

<!-- Displays the conversation between users -->
<% usermessage.forEach(msg => { %>
    <p>Message: <%= msg %></p>
<% }) %>

<p>
    This messaging interface allows real-time communication between users. Messages
    are sent to the backend, stored in the database, and then rendered back on the
    screen. This ensures smooth and continuous interaction between users.
</p>
```

Files: Frontend/views/loginOTP.ejs

```
<!-- OTP verification for secure login -->
<form action="/loginVerify" method="post">
    <input type="hidden" name="email" value="<%= email %>" />
    <input type="text" name="otp" placeholder="Enter OTP" required />
    <input type="submit" value="Verify OTP" />
</form>

<p>
    This step enhances security by introducing two-factor authentication during
login.
    The user must enter the correct OTP received via email to proceed further.
```

```
</p>
```

Files: Frontend/views/otpVerification.ejs

```
<!-- OTP verification during registration -->
<form action="/otpVerification" method="post">
    <input type="hidden" name="email" value="<%= email %>" />
    <input type="text" name="otp" placeholder="Enter OTP" required />
    <input type="submit" value="Verify OTP" />
</form>

<p>
    This ensures that only valid users can create an account. By verifying the email
    through OTP, the system prevents fake or unauthorized registrations.
</p>
```

Files: Frontend/views/aboutus.ejs

```
<!-- Provides overall description of the platform -->
<h1>About TradeLearn Platform</h1>

<p>
    TradeLearn is a collaborative platform designed to connect users who want to
    exchange skills and learn from each other. It provides a simple and interactive
    environment for users to explore opportunities and grow together.
</p>

<p>
    The platform integrates multiple features such as user search, messaging system,
    and OTP-based authentication. These features ensure both usability and security
    while maintaining a smooth user experience.
</p>

<p>
    Overall, this module demonstrates the integration of frontend forms, backend
    processing, and database interaction. It highlights how user inputs are handled,
    processed, and rendered dynamically to create a complete web application flow.
</p>
```